

MIT Art Design and Technology University's
MIT School of Computing, Pune
Department of Computer Science and Engineering
B. Tech Third Year **A.Y.2025-26**



Machine Learning Lab

Enrollment No: _____

Division: _____

Roll No: _____

Name: _____

Experiment No.: 10

Title: Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

Theory:	An Artificial Neural Network (ANN) is a computational model inspired by the working of biological neurons in the human brain. It consists of interconnected nodes (neurons) arranged in layers — an input layer, one or more hidden layers, and an output layer. Each connection between neurons has an associated weight, which determines the strength and direction of influence between the connected neurons. The learning process in an ANN involves adjusting these weights to minimize the difference between the actual output and the desired output. This adjustment is carried out using the Backpropagation Algorithm, which is one of the most popular and efficient supervised learning algorithms for training feedforward neural networks. Backpropagation Algorithm Steps: 1. Forward Propagation: ○ Inputs are passed through the network layer by layer. ○ Each neuron computes a weighted sum of its inputs, applies an activation function (e.g., ReLU or Sigmoid), and passes the result forward. ○ The output obtained is compared with the target output. 2. Error Calculation: ○ The difference between the target output and the network's output is computed using a loss function (e.g., Mean Squared Error or Cross-Entropy Loss). 3. Backward Propagation: ○ The calculated error is propagated backward through the network. ○ Partial derivatives of the error with respect to each weight are calculated using the chain rule of calculus. ○ This helps identify how much each weight contributed to the overall error.
----------------	--

4. Weight Update:

- Weights are updated to reduce the error using the gradient descent formula:

$$w_{new} = w_{old} - \eta \frac{\partial E}{\partial w}$$

where η is the learning rate, and $\frac{\partial E}{\partial w}$ is the gradient of the error with respect to the weight.

5. Iteration:

- Steps 1–4 are repeated for multiple **epochs** until the error is minimized or the model achieves the desired accuracy.

Activation Functions:

- Sigmoid:** $f(x) = \frac{1}{1+e^{-x}}$
- ReLU (Rectified Linear Unit):** $f(x) = \max(0, x)$
- Softmax:** Converts outputs into probability distribution for multi-class problems.

Applications:

- Image and speech recognition
- Medical diagnosis
- Stock price prediction
- Natural language processing

MIT Art Design and Technology University's
MIT School of Computing, Pune
Department of Computer Science and Engineering
B. Tech Third Year A.Y.2025-26



Flowchart: (if any)	
Pseudo code or Program	<pre> import numpy as np import tensorflow as tf from tensorflow.keras.models import Sequential from tensorflow.keras.layers import Dense from sklearn.datasets import load_iris from sklearn.model_selection import train_test_split from sklearn.preprocessing import StandardScaler, OneHotEncoder # ----- # 1. Load and preprocess the dataset # ----- iris = load_iris() X = iris.data # features (4) y = iris.target.reshape(-1, 1) # labels (0, 1, 2) # One-hot encode labels encoder = OneHotEncoder(sparse_output=False) y = encoder.fit_transform(y) # Normalize input features </pre>

```

scaler = StandardScaler()

X = scaler.fit_transform(X)

# Train-test split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# -----

# 2. Build ANN Model

# -----

model = Sequential([

    Dense(10, input_dim=4, activation='relu'), # Hidden layer 1

    Dense(8, activation='relu'),             # Hidden layer 2

    Dense(3, activation='softmax')           # Output layer (3 classes)

])

# -----

# 3. Compile Model

# -----

model.compile(optimizer='adam',

               loss='categorical_crossentropy',

               metrics=['accuracy'])

# -----

# 4. Train Model (Backpropagation happens here)

# -----

history = model.fit(X_train, y_train,

```

```

        epochs=100,

        batch_size=8,

        validation_split=0.2,

        verbose=1)

# -----

# 5. Evaluate Model

# -----

loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

print(f"\n Test Accuracy: {accuracy * 100:.2f}%")

# -----

# 6. Predict & Compare

# -----

predictions = model.predict(X_test)

predicted_classes = np.argmax(predictions, axis=1)

true_classes = np.argmax(y_test, axis=1)

print("\nTrue Labels:  ", true_classes)

print("Predicted Labels:", predicted_classes)

# -----

# 7. Plot training curves

# -----

import matplotlib.pyplot as plt

plt.plot(history.history['accuracy'], label='Train Accuracy')

```

	<pre>plt.plot(history.history['val_accuracy'], label='Validation Accuracy') plt.title('Model Accuracy') plt.xlabel('Epochs') plt.ylabel('Accuracy') plt.legend() plt.show()</pre>
Outputs	
Exercise for Practice	
Q1.	What is an Artificial Neural Network (ANN)?
Ans:	
Q2.	What is the role of the Backpropagation algorithm in ANN?
Ans	
Q3	What are activation functions, and why are they important?
Ans	

Q4.	How is the performance of an ANN evaluated?
Ans	
Conclusion	The Artificial Neural Network trained using the Backpropagation algorithm effectively minimizes the prediction error by iteratively updating the connection weights. This enables the model to learn patterns in the data and provide accurate predictions on test samples.
Date of Submission:	
Marks out of 10	Name and Sign of Subject Teacher
Remark:	